

DeFA: DEPENDENCY-GUIDED FAILURE ATTRIBUTION FOR LLM AGENTS

Bo Deng^{1,2} Xinlei Zheng² Yi Wei²
 Kang Zhou¹ Chongyang Tao² Renzhao Liang²
 Xuanren Chen² Lifan Guo¹ Chi Zhang¹

¹Qwen DianJin Team, Alibaba Cloud Computing

²Beihang University

ABSTRACT

Errors in LLM agent executions and their visible consequences can be separated by many steps, making decisive-error localization a matter of understanding both step content and step dependencies. We introduce DeFA, a dependency-guided framework for agent failure attribution. DeFA first combines protocol relations and semantic dependencies into an event dependency graph spanning the trajectory. It then identifies events that may violate task requirements and traces their sources and subsequent effects to construct a failure propagation graph. Finally, DeFA uses step evidence and the steps’ roles in failure propagation to identify the decisive error, responsible agent, and error category. To support long trajectories, DeFA partitions executions into segments and combines the current segment’s detailed content with summaries of the other segments, giving local diagnosis access to global execution context. Across Who&When and the Who&When Pro text subset, DeFA achieves the highest responsible-agent and exact step accuracy with all evaluated backbones, and the highest failure-mode accuracy among taxonomy-aligned methods on Pro. Further experiments on image and video trajectories demonstrate its applicability to multimodal failure attribution. Ablations support the contributions of segmentation, the event dependency graph, and the failure propagation graph. Using DeFA’s diagnostic feedback for skill evolution in Trace2Skill improves downstream task accuracy by 6–15 percentage points over the native pipeline, showing that the diagnoses can also support agent improvement on subsequent tasks.

1 INTRODUCTION

When an LLM agent fails a task, the final outcome rarely reveals which earlier decision was responsible. Agents interleave reasoning, tool use, and communication (Yao et al., 2023; Wu et al., 2024), producing trajectories in which an error and its visible consequences may be separated by many steps. Failure attribution identifies the responsible agent and the decisive error step (Zhang et al., 2025; Liu et al., 2026), providing specific evidence for debugging and subsequent improvement.

The difficulty lies in tracing how errors arise and propagate through execution to identify the decisive error behind downstream symptoms. Suppose an agent misreads a retrieved value, another agent uses it in a calculation, and the final answer repeats the incorrect result. The final answer reveals the failure, but the earlier interpretation may be the decisive error. Empirical studies likewise examine failure onset and propagation in agent trajectories (Zhao et al., 2026). Attribution must therefore examine what information a step received, how it used that information, and what happened afterward. This becomes harder when relevant evidence is distributed across a long trace (Liu et al., 2024).

Existing attribution frameworks use global judgments, sequential checks, or binary search (Zhang et al., 2025), as well as constraint validation, multi-agent investigation, and hierarchical diagnosis (Barke et al., 2026; Li et al., 2026; Wang et al., 2026). Our focus is to connect suspicious events with the execution that made them consequential. A local deviation must be examined together with its upstream sources and the later actions that adopted, amplified, or corrected its effects. Reconstructing these relations brings distributed evidence together to explain how the execution failed.

We introduce **DeFA**, a framework for **dependency-guided failure attribution** from recorded trajectories. DeFA first combines protocol relations and semantic dependencies into an event dependency graph spanning the trajectory. Building on this structure, it identifies events that may violate task requirements and traces their sources and subsequent effects to construct a failure propagation graph. It then combines trajectory content and propagation relations to identify the decisive error step, responsible agent, and error category. For long trajectories, DeFA uses segmented analysis, combining the current segment’s detailed content with summaries of the other segments so that local diagnosis can use both concrete step evidence and global execution context.

Against seven baselines, DeFA leads in responsible-agent and exact step accuracy on Who&When with all three backbones, improving exact step by 7.6–9.8 percentage points. On 300 Pro text trajectories, it also achieves the highest responsible-agent and exact step accuracy with all three backbones, as well as the highest failure-mode accuracy among taxonomy-aligned methods. On 100 multimodal trajectories, it improves exact step accuracy by 6.0–7.0 points across both backbones. Ablations show that segmentation, the event dependency graph, and the failure propagation graph each contribute to attribution accuracy. Removing propagation relations lowers exact step accuracy even when the adjudication set is held fixed, supporting their role in identifying the decisive error. Downstream experiments integrate DeFA’s diagnostic feedback into Trace2Skill, improving test accuracy over the native pipeline by 6–15 percentage points across human-written and model-generated initial skills and demonstrating its utility for agent improvement on subsequent tasks.

Our contributions are:

- We introduce a failure attribution framework that combines protocol relations and semantic dependencies into an event dependency graph to identify suspicious events, trace their contribution to task failure, and locate the decisive error and responsible agent.
- We develop a failure propagation graph that connects suspicious events with their upstream sources and downstream effects, tracing how errors are introduced, transmitted, and adopted to support decisive-error attribution.
- We demonstrate gains in precise attribution across text and multimodal evaluations, and show that DeFA’s diagnostic feedback improves downstream skill evolution by 6–15 percentage points over native Trace2Skill.

2 RELATED WORK

Execution trajectory evaluation and analysis. Agent evaluation includes response judging (Zheng et al., 2023), intermediate work and competency assessment through Agent-as-a-Judge and AgentDiagnose (Zhuge et al., 2025; Ou et al., 2025), and reasoning correctness and error localization through process supervision and ProcessBench (Lightman et al., 2024; Zheng et al., 2025). MAST characterizes design, coordination, and verification failures (Cemri et al., 2025); ToolEmu evaluates risk in emulated tool environments (Ruan et al., 2024); and Zhao et al. (2026) study coding-agent failure development and recovery.

Failure diagnosis and attribution frameworks. Who&When formalizes responsible-agent and decisive-step attribution with global, sequential, and binary-search judges (Zhang et al., 2025). TRAIL supplies a taxonomy and human-annotated diagnostic traces (Deshpande et al., 2025); Who&When Pro uses controlled error injection across modalities (Liu et al., 2026).

AgentRx supplies validated constraint violations to an attribution judge (Barke et al., 2026), VerifyMAS tests taxonomy-derived hypotheses (Qiao et al., 2026), and DeepDebug combines global reading, focused investigation, and cross-examination (Zhu et al., 2026). ErrorProbe detects local anomalies using a failure taxonomy and traces backward from observed symptoms to focus the investigation. Specialized agents then propose, investigate, and verify error hypotheses using executable evidence (Li et al., 2026). CockpitHAT performs hierarchical diagnosis for embodied multi-agent cockpits, integrating dialogue and other execution evidence with a panel of analysts. Its consensus procedure weighs analyst confidence and emphasizes failures with greater safety consequences (Wang et al., 2026). EDGE studies attribution of multiple error categories within a trajectory, combining a two-stage LLM judge for detection with counterfactual rollout analysis for interpreting errors and potential repairs (Hou et al., 2026).

Additional executions provide another source of evidence: AgenTracer constructs training supervision through counterfactual replay and fault injection (Zhang et al., 2026), FAMAS estimates suspiciousness from variation across repeated executions (Ge et al., 2026), and REFLECT and CausalFlow assess responsibility through patched replay or step-level interventions (Lin et al., 2026; Bonagiri et al., 2026).

Diagnostic feedback for agent improvement. Self-Refine iteratively revises outputs using feedback (Madaan et al., 2023); Reflexion carries verbal feedback into later trials (Shinn et al., 2023). Trace2Skill consolidates trajectory lessons into reusable skills (Ni et al., 2026), including for SpreadsheetBench tasks (Ma et al., 2024). We evaluate DeFA’s diagnostic feedback within this skill evolution process.

3 METHOD

DeFA constructs an event dependency graph to guide candidate recall, then traces candidate sources and effects in a failure propagation graph. Final adjudication compares the resulting steps and evidence to identify the decisive error. Figure 1 summarizes the pipeline.

3.1 PROBLEM FORMULATION

Following Who&When and Who&When Pro (Zhang et al., 2025; Liu et al., 2026), given task x and failed trajectory $\tau = (s_1, \dots, s_T)$, DeFA predicts the *decisive error step* $\hat{t}$: the earliest erroneous step whose correction would make execution succeed. Its responsible agent is $\hat{a} = a_{\hat{t}}$, where a_t is the actor recorded at s_t . The error category $\hat{y} \in \mathcal{Y}$ follows the Pro taxonomy, including hallucination and tool parameter/invoke error.

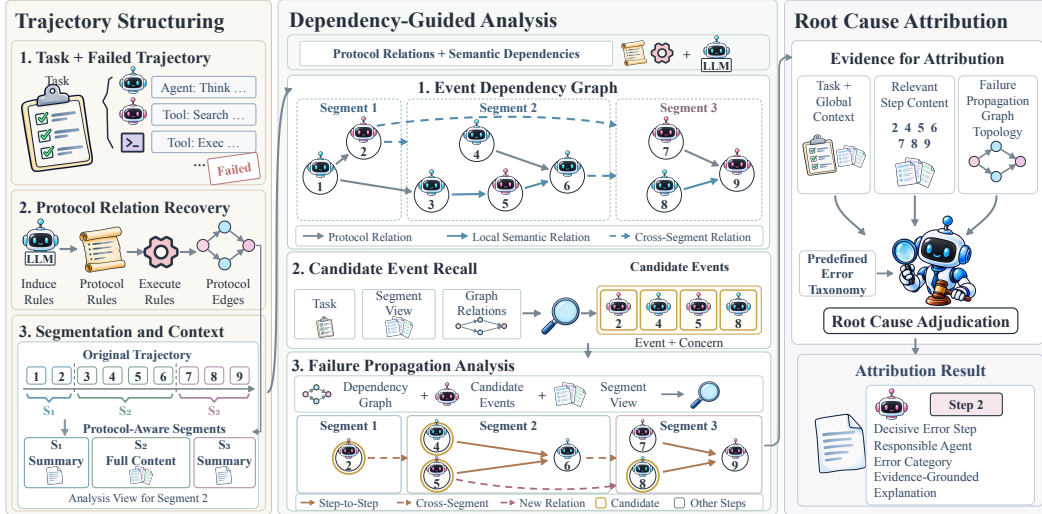


Figure 1: Overview of DeFA’s dependency-guided failure attribution pipeline.

3.2 EVENT DEPENDENCY GRAPH CONSTRUCTION

The event dependency graph represents each step as a node in $V = \{1, \dots, T\}$ and combines protocol relations with semantic dependencies. It records how steps pass and use instructions, results, and state.

Protocol relations capture interactions defined by the agent framework’s conventions, such as task delegation and its execution, or tool requests and their results. Semantic dependencies capture how later steps refer to or use earlier content, such as adopting a previous conclusion or using a retrieved value in a calculation.

Protocol relation recovery. DeFA first compresses oversized step content into a normalized trajectory $\tilde{\tau}$, preserving step identities and actors under instructions to retain facts, provenance, and action status. An LLM then induces protocol rules from message roles, formats, and interaction patterns. Each rule specifies source and target matching conditions, a pairing procedure, and the resulting relation. A deterministic interpreter scans $\tilde{\tau}$ to form the *protocol edge set* E_p : each edge (i, j) links a source step to a matched later step, such as a tool request to its result or a delegation to its execution. The interpreter supports next-match, all-match, FIFO, and LIFO pairing to accommodate different interaction patterns. These edges guide segmentation and complement semantic dependencies.

Segmentation and context. DeFA forms M consecutive segments $\mathcal{S} = (S_1, \dots, S_M)$ under step and content budgets, adjusting boundaries using E_p to preserve related interactions where possible. For multiple segments, it generates a narrative summary u_j of each. The analysis view $\mathcal{V}_j$ combines the full normalized content of S_j with the other summaries:

$$\mathcal{V}_j = (u_1, \dots, u_{j-1}, S_j, u_{j+1}, \dots, u_M). \quad (1)$$

This view limits context length while preserving local detail and broader execution: earlier summaries describe information sources, and later ones show whether information was used, corrected, or abandoned. Semantic dependency recovery, candidate recall, and failure propagation analysis each use this view per segment, retaining original step indices. Single-segment trajectories use the complete normalized trajectory directly.

Semantic dependency recovery. For each S_j , the model uses the task, $\mathcal{V}_j$, and relevant protocol edges to identify direct uses of earlier content, instructions, results, or state. Each semantic edge explains what its source supplied and how its target used it, such as a calculation using a value from an earlier observation. Local edges connect two steps in the detailed segment and merge into E_s^{local} . Cross-segment edges connect a step in S_j to a later segment S_k ($k > j$) whose summary indicates continued use of its information; these merge into E_s^{cross} . All edges point forward in execution order, forming

$$G_c = (V, \mathcal{S}, E_p \cup E_s^{\text{local}}, E_s^{\text{cross}}). \quad (2)$$

3.3 CANDIDATE EVENT RECALL

For each S_j , the recall model R examines every step using task requirements, $\mathcal{V}_j$, and event dependency graph relations $G_c^{(j)}$: step-to-step edges with an endpoint in S_j and cross-segment edges with a source in S_j or a target of S_j . It returns candidate records $\mathcal{Q}_j$, merged into $\mathcal{Q}$ with step set C :

$$\mathcal{Q}_j = R(x, \mathcal{V}_j, G_c^{(j)}), \quad \mathcal{Q} = \bigcup_{j=1}^M \mathcal{Q}_j, \quad C = \text{Steps}(\mathcal{Q}). \quad (3)$$

Each record retains the original step index, describes the step’s action or output, and explains its potential connection to failure using task requirements and trajectory evidence, such as information inconsistent with task constraints or an omitted necessary operation.

3.4 FAILURE PROPAGATION ANALYSIS

Using x , $\mathcal{V}_j$, $\mathcal{Q}_j$, and $G_c^{(j)}$, DeFA traces candidates’ upstream sources and how later execution continued, amplified, or corrected their effects. The model selects relevant edges from the event dependency graph and infers additional propagation edges from trajectory evidence. Each edge explains how source information or results were used and affected the task outcome. Step-to-step edges form E_f^{exact} ; edges targeting later segments form E_f^{cross} . Merging segment outputs yields the failure propagation graph G_f , whose step set includes candidates, exact edge endpoints, and cross-segment source steps:

$$G_f = (V_f, \mathcal{S}, E_f^{\text{exact}}, E_f^{\text{cross}}), \quad (4)$$

$$V_f = C \cup \text{Endpoints}_V(E_f^{\text{exact}}) \cup \text{Sources}_V(E_f^{\text{cross}}).$$

Thus, steps discovered through propagation relations join the initial candidates in V_f for final attribution.

3.5 ROOT CAUSE ATTRIBUTION

An LLM adjudicator J selects the decisive error from V_f using the task, failure propagation graph, and trajectory evidence view $\mathcal{Z}$. For a single segment, $\mathcal{Z}$ is the complete normalized trajectory; otherwise, it contains all segment summaries in execution order and the full normalized content of each step in V_f .

After identifying why the task failed, the adjudicator uses step content and propagation relations to compare plausible errors and their effects on the outcome. For each plausible root cause, it assesses whether a correct alternative was feasible at that point and whether substituting it, with later execution continuing from the corrected state, would turn failure into success. When several steps qualify, it compares how directly each explains the failure and the supporting evidence. It selects $\hat{t}$ and classifies that step’s own error under $\mathcal{Y}$:

$$(\hat{t}, \hat{y}) = J(x, \mathcal{Z}, G_f; \mathcal{Y}). \quad (5)$$

Here, $\hat{t} \in V_f$, $\hat{y} \in \mathcal{Y}$, and $\hat{a} = a_{\hat{t}}$. DeFA returns these predictions with an attribution explanation grounded in trajectory evidence.

Illustrative attribution. In an illustrative Who&When example, WebSurfer retrieves a film-ranking table at step 12, Assistant uses it to compare two rankings at step 16, and step 19 outputs the resulting count. The failure propagation graph connects these steps as $12 \rightarrow 16 \rightarrow 19$. Comparing the retrieved table with Assistant’s interpretation reveals that the table contains the needed entry, but Assistant omits it from the comparison. The missing film is Demon Slayer, whose inclusion changes the answer from five shared films to the correct count of six. DeFA attributes the failure to step 16, matching the benchmark annotation. The connected evidence identifies where the available information was used incorrectly.

Specifically, the retrieved domestic ranking places Demon Slayer tenth and Wonder Woman 1984 immediately below it. Assistant includes the latter in the domestic top ten, creating the mismatch used in the overlap calculation.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Benchmarks. We evaluate on Who&When (Zhang et al., 2025) and Who&When Pro (Liu et al., 2026). Who&When contains 184 annotated failure trajectories from algorithm-generated and hand-crafted multi-agent systems; we pool both subsets for evaluation. From Pro, we select 300 text trajectories spanning eight single- and multi-agent frameworks, stratified by framework and failure-mode group with additional coverage constraints on trajectory length and error position. All methods use the same evaluation trajectories.

Baselines and backbones. We compare DeFA with seven baselines: All-at-Once, Step-by-Step, and Binary Search from Who&When (Zhang et al., 2025); AgentRx (Barke et al., 2026); DeepDebug, the diagnostic component of AgentDebugX (Zhu et al., 2026); ErrorProbe (Li et al., 2026); and CockpitHAT (Wang et al., 2026). We evaluate every method with GLM-5.2, DeepSeek-V4-Pro, and Qwen3.8-Max on both text benchmarks. For each backbone, all methods use the same model for every LLM invocation, with temperature set to 0 and greedy decoding.

Metrics and scoring. We evaluate attribution along three dimensions: responsible agent, error step, and failure mode. **Agent** measures responsible-agent accuracy on trajectories from multi-agent frameworks. It uses all 184 Who&When trajectories and, following Pro’s official Who evaluation protocol, 89 Pro text trajectories, excluding Smolagents, ALFAgent, and MathChat. **Step** measures exact error-step localization accuracy; Step ± 1 and Step ± 3 allow deviations of up to one and three steps, respectively. These metrics use all 184 Who&When and all 300 Pro text trajectories. **Mode** measures failure-mode classification accuracy on all 300 Pro text trajectories. We compare the three Who&When judges and DeFA; the other methods do not adopt Pro’s failure-mode taxonomy and are therefore excluded from this comparison, with dashes shown in the tables.

We score the final prediction for each trajectory, evaluating each metric independently. Some Who&When Pro trajectories have multiple acceptable attribution answers. Following the official scoring rules, a prediction is correct on a metric if it matches any accepted answer on that axis. All methods’ step predictions are mapped to official coordinates before scoring.

Each accuracy is the number of correctly predicted trajectories divided by the total number evaluated for that metric. For Step $\pm k$ ($S_{\pm k}$ in tables, $k \in \{1, 3\}$), let $\hat{t}_i$ be the predicted step position and T_i the set of error positions accepted by the official scoring rules. A prediction is correct if it lies within k steps of any accepted position, giving the correctness indicator:

$$c_i^{(\pm k)} = \mathbf{1} \left[\min_{t \in T_i} |\hat{t}_i - t| \leq k \right]. \quad (6)$$

Tables bold the best results and underline the second-best distinct results within each backbone, with ties marked alike.

Table 1: Failure attribution accuracy on Who&When (%).

Method	GLM-5.2				DeepSeek-V4-Pro				Qwen3.8-Max			
	Agent	Step	$S_{\pm 1}$	$S_{\pm 3}$	Agent	Step	$S_{\pm 1}$	$S_{\pm 3}$	Agent	Step	$S_{\pm 1}$	$S_{\pm 3}$
All-at-Once	62.5	17.9	46.7	68.5	<u>62.5</u>	14.7	46.2	72.8	62.0	15.8	46.7	60.9
Step-by-Step	46.7	27.2	41.3	60.9	45.7	22.3	41.8	60.9	41.3	23.4	41.8	59.2
Binary Search	54.9	31.5	44.6	60.9	60.9	34.8	46.7	64.7	<u>64.1</u>	37.5	49.5	63.0
AgentRx	59.8	38.6	<u>51.6</u>	67.9	57.1	35.9	50.0	65.8	49.5	39.7	46.2	64.7
DeepDebug	58.7	38.6	50.0	<u>69.0</u>	58.7	<u>39.1</u>	<u>50.5</u>	67.9	55.4	36.4	<u>50.0</u>	70.1
ErrorProbe	59.8	<u>43.5</u>	<u>51.6</u>	65.2	57.6	36.4	44.0	61.4	59.8	39.1	47.3	65.2
CockpitHAT	<u>63.6</u>	42.9	<u>51.6</u>	65.2	56.5	38.6	47.3	65.2	60.9	<u>42.4</u>	48.4	58.7
DeFA (ours)	72.3	52.7	57.1	71.7	68.5	48.9	55.4	<u>69.0</u>	70.7	50.0	56.0	<u>69.0</u>

4.2 MAIN RESULTS ON WHO&WHEN

DeFA achieves the highest Agent and exact Step accuracy with all three backbones (Table 1). Exact Step improves by 7.6–9.8 percentage points over the strongest baseline per backbone, reaching 52.7% with GLM-5.2. The accompanying gains in Agent accuracy demonstrate its effectiveness in identifying the responsible agent and the decisive error step.

DeFA also leads in Step ± 1 with every backbone. Although the strongest baseline changes across models, DeFA consistently maintains its precise-localization advantage. These results support connecting step content with its upstream sources and downstream effects as an effective basis for diagnosing multi-agent failures.

4.3 MAIN RESULTS ON WHO&WHEN PRO

Table 2: Failure attribution accuracy on the Who&When Pro text subset (%).

Method	GLM-5.2					DeepSeek-V4-Pro					Qwen3.8-Max				
	Agent	Step	$S_{\pm 1}$	$S_{\pm 3}$	Mode	Agent	Step	$S_{\pm 1}$	$S_{\pm 3}$	Mode	Agent	Step	$S_{\pm 1}$	$S_{\pm 3}$	Mode
All-at-Once	67.4	69.7	80.0	89.3	<u>28.0</u>	71.9	68.3	79.3	90.3	<u>27.3</u>	<u>69.7</u>	<u>71.7</u>	<u>83.3</u>	91.7	26.3
Step-by-Step	66.3	69.0	77.3	87.7	<u>27.7</u>	69.7	66.0	74.7	86.7	24.0	75.3	<u>71.7</u>	78.3	87.0	<u>27.0</u>
Binary Search	60.7	64.3	74.0	91.7	25.0	56.2	68.3	83.7	92.7	25.0	68.5	70.3	83.0	92.0	25.7
AgentRx	66.3	68.0	<u>81.7</u>	89.3	—	67.4	69.0	81.7	<u>93.3</u>	—	59.6	63.7	83.7	92.3	—
DeepDebug	64.0	68.7	79.3	91.0	—	67.4	69.0	80.0	91.7	—	65.2	68.0	78.3	90.0	—
ErrorProbe	70.8	70.3	79.7	92.3	—	62.9	66.7	74.3	87.7	—	67.4	68.0	79.0	91.3	—
CockpitHAT	<u>77.5</u>	<u>72.0</u>	81.3	<u>93.7</u>	—	<u>74.2</u>	<u>72.3</u>	<u>82.7</u>	94.0	—	68.5	69.3	80.7	94.0	—
DeFA (ours)	85.4	80.3	89.3	95.3	41.0	79.8	76.3	80.0	92.7	34.0	75.3	77.7	<u>83.3</u>	<u>92.7</u>	37.7

On Pro text, DeFA achieves the highest Agent and exact Step accuracy with all three backbones (Table 2). Exact Step gains over the strongest baseline range from 4.0 to 8.3 percentage points, reaching 80.3% with GLM-5.2. This consistent performance on a benchmark spanning eight agent frameworks further supports DeFA’s applicability to diverse agent executions.

DeFA also leads taxonomy-aligned methods in Mode accuracy, reaching 34.0–41.0% and improving over the strongest comparable baseline by 6.7–13.0 percentage points. Together, the Step and Mode results demonstrate its effectiveness in both locating errors and characterizing their nature.

4.4 TRAJECTORY LENGTH AND LLM CALLS

Localization gains across trajectory lengths. Figure 2a,b shows exact Step accuracy across trajectory lengths with GLM-5.2. Accuracy generally declines as trajectories grow longer, but DeFA outperforms the baselines in every length interval on both benchmarks. The largest gains occur at intermediate lengths: DeFA reaches 45.8% on Who&When trajectories of 11–32 steps, exceeding the strongest baseline by 12.5 percentage points, and improves accuracy by 18.3 points on Pro trajectories of 6–8 steps (67.6% versus 49.3%). These results show that DeFA’s overall improvement spans different trajectory lengths. Nevertheless, its accuracy remains only 17.2% in the longest Who&When interval of 33–130 steps despite leading the baselines, indicating that exact localization in long trajectories remains challenging.

LLM calls and Step accuracy. We compare each method’s mean number of LLM calls per trajectory with its exact Step accuracy (Figure 2c,d). DeFA achieves the highest Step accuracy on both benchmarks. On Who&When, DeFA reaches 52.7% with 8.73 calls on average, compared with 43.5% and 6.93 calls for the strongest baseline, ErrorProbe. On Pro, DeFA reaches 80.3% with 8.73 calls, compared with 72.0% and 6.35 calls for the strongest baseline, CockpitHAT.

4.5 EXTENSION TO MULTIMODAL TRAJECTORIES

To assess DeFA’s applicability beyond text-only trajectories, we further evaluate its attribution performance on multimodal trajectories containing image and video evidence. We construct multimodal inputs by combining images or video frames with the text of their corresponding steps, preserving this alignment during context construction so that DeFA can attribute failures using both textual and visual evidence.

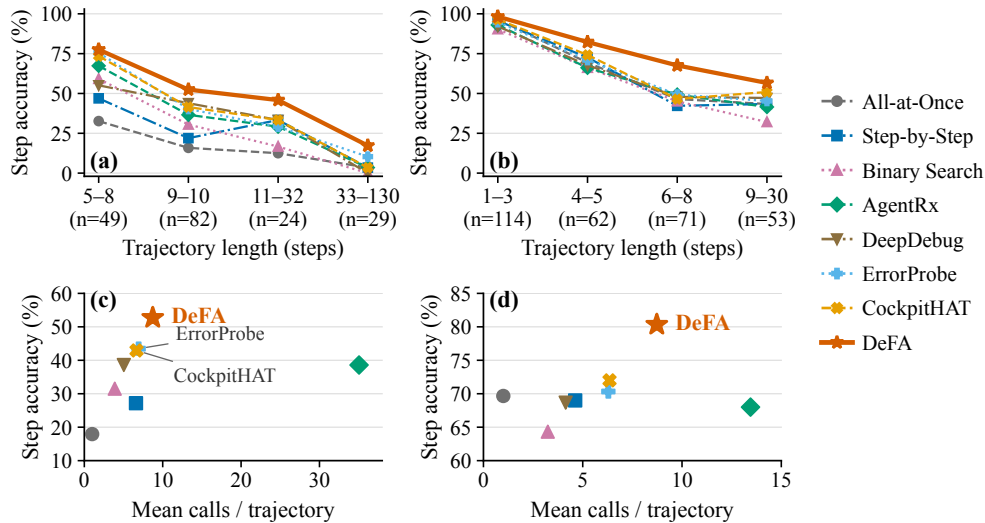


Figure 2: GLM-5.2 exact Step accuracy on Who&When (left) and Pro text (right), by (a,b) trajectory length and (c,d) mean LLM calls per trajectory.

We evaluate all eight methods with Qwen3.8-Max and Kimi-K3 on a stratified sample of 70 image and 30 video trajectories from Who&When Pro. All methods use the same benchmark visual inputs and retain their respective attribution procedures. Step, Step ± 1 , Step ± 3 , and Mode use all 100 trajectories; following the official Who evaluation protocol, Agent accuracy is computed on the 17 trajectories from multi-agent frameworks.

Table 3: Multimodal failure attribution accuracy on Who&When Pro (%).

Method	Qwen3.8-Max					Kimi-K3				
	Agent	Step	S ± 1	S ± 3	Mode	Agent	Step	S ± 1	S ± 3	Mode
All-at-Once	58.8	68.0	80.0	88.0	<u>42.0</u>	<u>70.6</u>	68.0	79.0	89.0	<u>40.0</u>
Step-by-Step	<u>70.6</u>	64.0	74.0	87.0	33.0	58.8	61.0	69.0	81.0	32.0
Binary Search	<u>70.6</u>	69.0	79.0	89.0	37.0	58.8	67.0	80.0	91.0	35.0
AgentRx	64.7	64.0	74.0	85.0	—	58.8	70.0	83.0	88.0	—
DeepDebug	58.8	69.0	<u>82.0</u>	92.0	—	64.7	65.0	<u>82.0</u>	91.0	—
ErrorProbe	58.8	66.0	78.0	89.0	—	47.1	70.0	81.0	89.0	—
CockpitHAT	<u>70.6</u>	<u>73.0</u>	<u>82.0</u>	<u>91.0</u>	—	64.7	<u>72.0</u>	81.0	87.0	—
DeFA (ours)	76.5	80.0	84.0	89.0	50.0	82.4	78.0	<u>82.0</u>	<u>90.0</u>	49.0

DeFA retains its attribution advantage on multimodal trajectories, leading Agent and exact Step with both backbones and Mode among taxonomy-aligned methods (Table 3). Step reaches 80.0% with Qwen3.8-Max and 78.0% with Kimi-K3, exceeding CockpitHAT by 7.0 and 6.0 percentage points; Mode reaches 50.0% and 49.0%.

These results extend DeFA’s effectiveness to executions with visual evidence. Keeping images and video frames aligned with their steps enables dependency and propagation analysis to relate visual observations to subsequent decisions and errors. Gains in both localization and classification support the framework’s applicability to multimodal failure diagnosis.

4.6 COMPONENT ABLATIONS

We evaluate five ablations with GLM-5.2 on Who&When. Removing segmentation treats each trajectory as one segment; removing the event dependency graph omits dependency context; removing the failure propagation graph skips propagation analysis. A fourth variant removes both graphs. The fifth removes only failure propagation edges and their explanations from final adjudication, retaining all steps and their evidence. All graph variants retain segmentation. Coverage is the fraction of trajectories whose annotated error step is in the final adjudication set A .

Table 4: Component ablations on Who&When with GLM-5.2 (%).

Setting	Coverage	Step	Agent	Step ± 1
Full DeFA	83.2	52.7	72.3	57.1
w/o segmentation	75.0	44.6	60.9	53.3
w/o failure propagation graph	78.8	40.8	56.5	51.6
w/o event dependency graph	77.7	42.4	58.2	49.5
w/o both graphs	73.4	36.4	53.8	46.2
w/o failure propagation edges	83.2	40.2	57.1	48.9

Full DeFA leads all four metrics (Table 4). Removing segmentation lowers Step from 52.7% to 44.6%. Without the event dependency graph, Coverage falls to 77.7% and Step to 42.4%, supporting dependency context for candidate recall and tracing how steps use earlier information. Removing the failure propagation graph reduces Step to 40.8%; removing both graphs to 36.4%. Removing only propagation edges retains the same adjudication set and 83.2% Coverage, but lowers Step to 40.2%, a 12.5-point drop. This supports using propagation relations to trace how errors are introduced, transmitted, and adopted when locating the decisive error.

4.7 DOWNSTREAM UTILITY: SKILL IMPROVEMENT

We integrate DeFA into Trace2Skill (Ni et al., 2026). Trace2Skill analyzes agent execution trajectories to identify the causes of failure in failed trajectories and patterns in successful trajectories, using these analyses to evolve reusable skills. We replace the failed-trajectory analysis module in its Combined setting with DeFA.

Experimental setup. We use 100 SpreadsheetBench-Verified (Ma et al., 2024) tasks for skill evolution and another 100 for testing. We evaluate Kimi-K2.5 and Qwen3.5-Plus, each with a human-written `xlsx` skill or a model-generated initial skill, yielding four settings. Following the original Trace2Skill setup, skill evolution and testing use the same model.

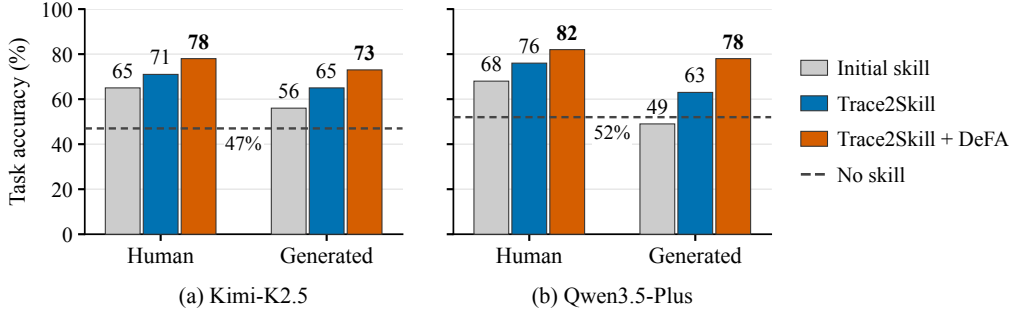


Figure 3: Test accuracy after skill evolution on 100 SpreadsheetBench-Verified tasks. Human and Generated denote human-written and model-generated initial skills; dashed lines denote no skill.

Effect on skill evolution. With DeFA feedback, test accuracy improves over native Trace2Skill in all four settings by 6–15 percentage points, averaging 9.0 points (Figure 3). The largest gain occurs with Qwen3.5-Plus and a model-generated initial skill, where accuracy rises from 63% to 78%. Both models benefit under each initialization, showing that DeFA’s diagnostic feedback can improve human-written and model-generated skills for subsequent tasks. Trajectory diagnosis and its full analytical outputs may also provide feedback for broader agent self-improvement and continual evolution.

5 CONCLUSION AND LIMITATIONS

We introduced DeFA, which traces agent failures through step dependencies. It combines protocol relations and semantic dependencies into an event dependency graph spanning the trajectory to capture how steps use earlier information. DeFA then identifies events that may violate task requirements and traces their sources and effects to construct a failure propagation graph. Final attribution uses this connected evidence to compare how steps introduce, transmit, or adopt errors and identify the decisive error and responsible agent.

Experiments on Who&When and Who&When Pro demonstrate precise attribution gains across backbones, trajectory lengths, and text and multimodal inputs. Ablations support segmentation, the event dependency graph, and the failure propagation graph. DeFA’s diagnostic feedback also improves test accuracy after Trace2Skill skill evolution by an average of 9.0 percentage points over the native pipeline, showing that the diagnoses can guide skill improvements for subsequent tasks.

Limitations. DeFA’s semantic dependency recovery and failure propagation analysis depend on the model’s trajectory interpretation; missing or misidentified relations can affect attribution, especially in long executions with dispersed evidence. Downstream utility has been evaluated for spreadsheet skill evolution; effectiveness in other domains and forms of agent improvement remains to be assessed. Future work can improve relation identification in long trajectories and broaden the applications of diagnostic feedback.

AI USE STATEMENT

We used generative AI tools to assist with manuscript editing and translation, and with writing and revising parts of the research code. All AI-assisted text and code were reviewed and checked by the authors. The authors take full responsibility for the final manuscript, implementation, and reported results.

REFERENCES

- Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. AgentRx: Diagnosing AI agent failures from execution trajectories. *arXiv preprint arXiv:2602.02475*, 2026. doi: 10.48550/arXiv.2602.02475. URL <https://arxiv.org/abs/2602.02475>. Accepted to Findings of EMNLP 2026; arXiv version 2.
- Akash Bonagiri, Devang Borkar, Gerard Janno Anderias, Setareh Rafatirad, and Houman Homaoun. CausalFlow: Causal attribution and counterfactual repair for LLM agent failures. *arXiv preprint arXiv:2605.25338*, 2026. doi: 10.48550/arXiv.2605.25338. URL <https://arxiv.org/abs/2605.25338>.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei A. Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems*, volume 38, 2025. doi: 10.52202/085713-4082. URL <https://arxiv.org/abs/2503.13657>.
- Darshan Deshpande, Varun Gangal, Hersh Mehta, Jitin Krishnan, Anand Kannappan, and Rebecca Qian. TRAIL: Trace reasoning and agentic issue localization. *arXiv preprint arXiv:2505.08638*, 2025. doi: 10.48550/arXiv.2505.08638. URL <https://arxiv.org/abs/2505.08638>.
- Yu Ge, Linna Xie, Zhong Li, Yu Pei, and Tian Zhang. Spectrum-based failure attribution for multi-agent systems. *Proceedings of the ACM on Software Engineering*, 3(FSE):1888–1909, 2026. doi: 10.1145/3797113. URL <https://doi.org/10.1145/3797113>.
- Jun Hou, Priya Pitre, Yi Fang, and Xuan Wang. EDGE: Error dependency graph-guided multi-error attribution in multi-agent LLM systems. *arXiv preprint arXiv:2609.01360*, 2026. URL <https://arxiv.org/abs/2609.01360>.
- Jiazheng Li, Emine Yilmaz, Bei Chen, and Thu Le. Towards self-improving error diagnosis in multi-agent systems. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 2063–2077. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.98. URL <https://aclanthology.org/2026.findings-acl.98/>.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=v8L0pN6EOi>.
- Xiaofeng Lin, Yingxu Wang, Tung Sum Thomas Kwok, Daniel Guo, Sahil Arun Nale, Charles Fleming, and Guang Cheng. REFLECT: Intervention-supported error attribution for silent failures in LLM agent traces. *arXiv preprint arXiv:2606.09071*, 2026. doi: 10.48550/arXiv.2606.09071. URL <https://arxiv.org/abs/2606.09071>.
- Jiale Liu, Huajun Xi, Shaokun Zhang, Yifan Zeng, Tianwei Yue, Chi Wang, Jian Kang, Qingyun Wu, and Huazheng Wang. Who&When Pro: Can LLMs really attribute failures in AI agents? *arXiv preprint arXiv:2607.09996*, 2026. doi: 10.48550/arXiv.2607.09996. URL <https://arxiv.org/abs/2607.09996>.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tac1a-00638. URL <https://aclanthology.org/2024.tac1-1.9/>.

-
- Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. SpreadsheetBench: Towards challenging real world spreadsheet manipulation. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-3007. URL https://papers.nips.cc/paper_files/paper/2024/hash/ac840df270ac537dd74530a15c332684-Abstract-Datasets_and_Benchmarks_Track.html.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.52202/075280-2019. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. Trace2Skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026. doi: 10.48550/arXiv.2603.25158. URL <https://arxiv.org/abs/2603.25158>.
- Tianyue Ou, Wanyao Guo, Apurva Gandhi, Graham Neubig, and Xiang Yue. AgentDiagnose: An open toolkit for diagnosing LLM agent trajectories. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 207–215. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-demos.15. URL <https://aclanthology.org/2025.emnlp-demos.15/>.
- Hezhe Qiao, Hanghang Tong, Ee-Peng Lim, Bing Liu, and Guansong Pang. VerifyMAS: Hypothesis verification for failure attribution in LLM multi-agent systems. *arXiv preprint arXiv:2605.17467*, 2026. doi: 10.48550/arXiv.2605.17467. URL <https://arxiv.org/abs/2605.17467>.
- Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=GEcwtMklUa>.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.52202/075280-0377. URL https://papers.nips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.
- Wei Wang, Shuanghe Liu, Zhu Zhuo, Jiaqi Zhong, Xiaozhao Zhao, Xiaojie Zuo, and Jie Su. CockpitHAT: Dependency-graph-driven hierarchical attribution for embodied multi-agent cockpits. *arXiv preprint arXiv:2608.01805*, 2026. doi: 10.48550/arXiv.2608.01805. URL <https://arxiv.org/abs/2608.01805>.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=BAakY1hNKS>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. AgenTracer: Who is inducing failure in the LLM agentic systems? In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/134ed7a477770f227f12450ef0cbb8f4-Abstract-Conference.html.

-
- Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 76583–76599. PMLR, 2025. URL <https://proceedings.mlr.press/v267/zhang25cq.html>.
- Xiangxin Zhao, Han Li, Shuaiting Li, Tianyi Zhao, Earl T. Barr, Federica Sarro, and He Ye. Failure as a process: An anatomy of CLI coding agent trajectories. *arXiv preprint arXiv:2607.09510*, 2026. doi: 10.48550/arXiv.2607.09510. URL <https://arxiv.org/abs/2607.09510>.
- Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. ProcessBench: Identifying process errors in mathematical reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1009–1024. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.50. URL <https://aclanthology.org/2025.acl-long.50/>.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.52202/075280-2020. URL <https://arxiv.org/abs/2306.05685>.
- Kunlun Zhu, Xuyan Ye, Zhiguang Han, Yuchen Zhao, Bingxuan Li, Weijia Zhang, Muxin Tian, Xiangru Tang, Pan Lu, James Zou, Jiaxuan You, and Heng Ji. AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents. *arXiv preprint arXiv:2607.18754*, 2026. doi: 10.48550/arXiv.2607.18754. URL <https://arxiv.org/abs/2607.18754>.
- Mingchen Zhuge, Changsheng Zhao, Dylan R. Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. Agent-as-a-judge: Evaluate agents with agents. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 80569–80611. PMLR, 2025. URL <https://proceedings.mlr.press/v267/zhuge25a.html>.